

Informe Técnico — Sistema de Gestión de Contenido "StreamFlow"

Integrantes: Pablo A. Guamán L. — Jhonatan Gualán **Asignatura:** Programación Orientada a Objetos
Institución: Universidad Técnica Particular de Loja (UTPL)

1. Introducción

Este informe sustenta las decisiones de diseño tomadas para el sistema "StreamFlow", justificando el cumplimiento de los cinco principios SOLID, la separación de capas de persistencia y los patrones de diseño aplicados. Cada sección responde directamente a un criterio de la rúbrica de evaluación.

2. Cumplimiento de principios SOLID (30%)

2.1 SRP — Principio de Responsabilidad Única

Cada clase tiene una única razón para cambiar:

- **Contenido y sus subclases** (model): solo conocen su propio estado y cómo describirse/reproducirse. No saben nada de SQL ni de consola.
- **ContenidoDAOSQLite / UsuarioDAOSQLite** (dao): son las únicas clases del proyecto que contienen sentencias SQL. Si cambia el motor de base de datos, solo se tocan estas dos clases.
- **SuscripcionServiceImpl / RecomendacionServiceImpl** (service): contienen exclusivamente reglas de negocio (cálculo de costos, filtrado de catálogo), sin acceso directo a la base de datos ni a la consola.
- **StreamFlowView** (view): solo imprime lo que recibe, no decide nada.
- **StreamFlowController** (controller): solo orquesta llamadas entre las demás capas.

Esta separación permite que, si el profesor pide "cambia SQLite por otra base de datos" o "cambia la consola por una interfaz gráfica", el cambio quede contenido en una sola capa, sin efectos colaterales en el resto del sistema.

2.2 OCP — Abierto/Cerrado

El sistema está preparado para agregar Podcast o AudioLibro **sin modificar una sola línea** de SuscripcionServiceImpl, RecomendacionServiceImpl ni StreamFlowController, porque:

1. Todos operan sobre la referencia abstracta `Contenido` (o `List<Contenido>`), nunca sobre tipos concretos.
2. `reproducir()` y `obtenerDetalles()` son polimórficos: cada nueva subclase simplemente implementa su propia versión.
3. No existe ningún `if (contenido instanceof Pelicula)` ni `switch` sobre tipo en la capa de lógica de negocio. El único lugar donde se distingue el tipo concreto es dentro de `ContenidoDAOSQLite.mapearContenido()`, y esto es deliberado: **el mapeo objeto-relacional (leer una fila de SQLite y decidir qué objeto Java construir) es responsabilidad exclusiva del DAO**, no una violación de OCP en la capa de negocio. Es el mismo patrón que usaría un ORM como Hibernate internamente.

Para agregar `Podcast`, el trabajo sería: crear la clase `Podcast` `extends Contenido`, añadir sus columnas a la tabla `contenido`, y agregar un `case "PODCAST"` en `mapearContenido()`. Ningún archivo de `service` o `controller` cambia.

2.3 LSP — Sustitución de Liskov

Cualquier método que reciba o devuelva `Contenido` funciona correctamente sin importar si el objeto real es `Pelicula`, `Serie` o `Documental`:

- `Usuario.agregarFavorito(Contenido c)` acepta cualquier subtipo.
- `RecomendacionServiceImpl.recomendarPorGenero(...)` filtra una `List<Contenido>` heterogénea sin conocer los tipos concretos.
- El test `listaPolimorficaDebeProcesarseSinConocerElTipoConcreto` demuestra esto explícitamente: construye una lista mixta y la recorre invocando `reproducir()` y `obtenerDetalles()` sobre la referencia `Contenido`, sin que ninguna subclase rompa el comportamiento esperado (ninguna lanza una excepción inesperada, ninguna devuelve `null` o vacío).

Ninguna subclase restringe precondiciones ni debilita postcondiciones respecto a `Contenido`: todas garantizan que `obtenerDetalles()` retorna una cadena no vacía y que `reproducir()` no falla nunca. Por eso son verdaderamente sustituibles.

2.4 ISP — Segregación de Interfaces

En vez de una interfaz `Repositorio` gigante con veinte métodos, se definió `Crud<T>` con exactamente cinco operaciones genéricas (`crear`, `obtenerPorId`, `listarTodos`, `actualizar`, `eliminar`). `ContenidoDAO` y `UsuarioDAO` extienden `Crud<T>` y quedan abiertas para añadir métodos propios (por ejemplo, `buscarPorGenero` en el futuro) sin forzar a otras entidades a implementarlos.

De igual forma, `Reproducible` se separó de `Contenido` como interfaz independiente: si en el futuro se agrega una entidad que **no** deba reproducirse (por ejemplo, un Anuncio publicitario), no estará obligada a implementar `reproducir()`.

2.5 DIP — Inversión de Dependencias

`StreamFlowController` depende únicamente de las interfaces `ContenidoDAO`, `UsuarioDAO`, `SuscripcionService` y `RecomendacionService` — nunca de `ContenidoDAOSQLite` ni de `SuscripcionServiceImpl` directamente. Las implementaciones concretas se construyen e inyectan **solo en** `Main`, que actúa como composition root.

Esto significa que el controlador y los servicios podrían probarse de forma aislada usando un `ContenidoDAO` falso (un *stub* o *mock*) implementando la misma interfaz, sin tocar SQLite.

3. Persistencia (20%)

La capa de persistencia está completamente aislada en el paquete `dao`:

- **`ConexionSQLite`** centraliza la conexión JDBC (patrón *Singleton*) y la creación del esquema (`CREATE TABLE IF NOT EXISTS`), evitando conexiones duplicadas y garantizando que el esquema exista antes de cualquier operación.
- **`ContenidoDAOSQLite` y `UsuarioDAOSQLite`** implementan sus respectivas interfaces (`ContenidoDAO`, `UsuarioDAO`) usando `PreparedStatement` en todas las operaciones, lo que previene inyección SQL y asegura la integridad de los datos.
- Las claves primarias son `INTEGER PRIMARY KEY AUTOINCREMENT`, y el id generado se recupera con `getGeneratedKeys()` y se asigna de vuelta al objeto Java, manteniendo sincronizados el estado en memoria y en disco.
- Ninguna clase fuera de `dao` abre una `Connection` o escribe SQL: `service` y `controller` solo conocen las interfaces `ContenidoDAO/UsuarioDAO` (refuerza DIP y SRP simultáneamente).

Esta separación es lo que permite, por ejemplo, sustituir SQLite por PostgreSQL en un entorno de producción cambiando únicamente las clases del paquete `dao`, sin tocar `model`, `service`, `view` ni `controller`.

4. Documentación (10%)

4.1 Patrones de diseño utilizados

- **Singleton** (`ConexionSQLite`): garantiza una única instancia de conexión compartida por toda la aplicación.
- **DAO (Data Access Object)**: separa la lógica de acceso a datos de la lógica de negocio (`ContenidoDAO`, `UsuarioDAO`).
- **MVC (Model-View-Controller)**: `model` (estado del dominio), `view` (presentación por consola), `controller` (coordinación).
- **Strategy implícito vía polimorfismo**: cada subtipo de `Contenido` encapsula su propio algoritmo de reproducción y presentación, evitando condicionales.
- **Dependency Injection manual** (constructor injection): `Main` construye las dependencias concretas y las inyecta en `StreamFlowController` a través de sus constructores, sin frameworks externos.

4.2 Diagramas UML

Ver el archivo `diagramas_uml.md`, que contiene el diagrama de clases completo y dos diagramas de secuencia (reproducción de contenido y cálculo de suscripción) en formato Mermaid, listos para exportar a `.png` y para servir de base a un archivo `.dia`.

5. Conclusión

El diseño de "StreamFlow" prioriza la extensibilidad (OCP), el desacoplamiento (DIP, ISP) y la solidez de los contratos de herencia (LSP), apoyado en una separación de capas estricta (SRP, MVC, DAO). Cada decisión de diseño documentada aquí tiene una justificación técnica verificable en el código fuente.